

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

Отчет
по лабораторной работе № 4
по курсу «Арифметические и логические основы вычислительной
техники»
на тему «Бинарное дерево поиска»

Выполнили
студенты группы 24ВВВ2:

Макаров Я.С.

Бегичев Д.Д.

Родионов К.Е.

Приняли

Юрова О.В.

Деев М.В.

Пенза, 2025

Цель работы

Научиться работать с бинарными деревьями.

Лабораторное задание

1. Реализовать алгоритм поиска вводимого с клавиатуры значения в уже созданном дереве.
2. Реализовать функцию подсчёта числа вхождений заданного элемента в дерево.
3. *Изменить функцию добавления элементов для исключения добавления одинаковых символов.
4. *Оценить сложность процедуры поиска по значению в бинарном дереве.

Описание метода решения

Данная программа реализует бинарное дерево поиска (BST). Первая версия позволяет дубликаты, добавляя их в левое поддерево, вторая версия включает защиту от дубликатов через предварительный поиск перед вставкой. Основной метод решения основан на рекурсивных алгоритмах: вставка по принципу "меньшие - влево, большие - вправо", поиск за $O(h)$, где h - высота дерева. Подсчет вхождений требует полного обхода $O(n)$. Визуализация выполнена вертикальным выводом с отступами по уровням. Ключевое отличие между версиями - обработка дубликатов, влияющая на структуру дерева и производительность операций.

Теоретическая сложность поиска в бинарном дереве

- Время поиска зависит от высоты дерева h .
- Если дерево сбалансировано, высота будет порядка $O(\log n)$, где n — количество узлов.
- В худшем случае (выраженном в виде цепочки, например все элементы в одном поддереве), высота — $O(n)$, и поиск становится линейным по времени.

Псевдокод

1 задание

алг бинарное_дерево_поиска

дано

- пустое бинарное дерево root
- последовательность чисел для вставки и поиска

надо

- построить дерево из чисел
- искать элементы по значению
- выводить структуру дерева

нач цел

root := пусто

line := строка для ввода

token := указатель для разбора строки

D := число для вставки и поиска

// Функция проверки является ли строка числом (целым)

функция is_number(str)

нач

если str пустая или нулевой длины тогда вернуть 0

для каждого символа i в str

если символ не цифра и не минус в начале, вернуть 0

конец

вернуть 1

кон

// Рекурсивное создание дерева поиска (вставка)

функция CreateTree(r, data)

нач

если r пусто тогда

выделить память под новый узел p

если выделение памяти не удалось, вывести ошибку и завершить

p.data := data

p.left := пусто

p.right := пусто

вернуть p

иначе

если data > r.data тогда

```

        r.right := CreateTree(r.right, data)
    иначе
        r.left := CreateTree(r.left, data)
    вернуть r
кон

// Рекурсивный поиск элемента в дереве
функция search(r, key)
нач
    если r пусто или r.data = key
        вернуть r
    если key > r.data тогда
        вернуть search(r.right, key)
    иначе
        вернуть search(r.left, key)
кон

// Рекурсивный вывод дерева (консольное)
процедура print_tree(r, уровень)
нач
    если r пусто тогда вернуть
    print_tree(r.right, уровень + 1)
    вывести пробелы * уровень, затем r.data
    print_tree(r.left, уровень + 1)
кон

// Основная программа
нач
    установить локаль
    вывести приглашение для ввода чисел
    ввести строку line
    разбить строку на токены по пробелу/табуляции/переносу строки
    пока token не пусто
        если token равен "stop" или "in the end" тогда выйти из цикла
        если is_number(token) = 0 тогда вывести ошибку и пропустить
        иначе
            D := преобразовать token в число
            root := CreateTree(root, D)
        получить следующий token
    вывести построенное дерево вызовом print_tree(root, 0)
    ввести значения для поиска до команды "stop" или "in the end"
    для каждого введённого значения:
        если не число — вывести ошибку

```

иначе вызвать search(root, значение)
если найдено — вывести сообщение
иначе — вывести сообщение об отсутствии

кон

2 задание

алг бинарное_дерево_подсчет_вхождений

дано

- пустое бинарное дерево root
- последовательность чисел для построения дерева
- последовательность чисел для подсчёта вхождений

надо

- построить бинарное дерево поиска из чисел
- подсчитать, сколько раз заданный ключ встречается в дереве
- вывести дерево и результаты подсчёта

нач цел

root := пусто

line := вводимая строка

token := указатель для разбора строки

D, key := числа

count := счетчик

// Проверка, является ли строка числом

функция is_number(str)

нач

если str пустая или NULL тогда вернуть 0

для каждого символа i в str

если символ не цифра и не минус в начале строки - вернуть 0

конец

вернуть 1

кон

// Рекурсивное создание бинарного дерева поиска

функция CreateTree(r, data)

нач

если r пусто тогда

выделить память под узел p

если ошибка памяти - вывести сообщение и завершить

инициализировать p.data = data, p.left = NULL, p.right = NULL

```

        вернуть r
    иначе
        если data > r.data тогда
            r.right := CreateTree(r.right, data)
        иначе
            r.left := CreateTree(r.left, data)
        вернуть r
кон

// Рекурсивный подсчет вхождений ключа в дерево
функция count_occurrences(r, key)
нач
    если r пусто вернуть 0
    count := 1 если r.data = key иначе 0
    вернуть count + count_occurrences(r.left, key) + count_occurrences(r.right,
key)
кон

// Рекурсивный печат дерева
процедура print_tree(r, уровень)
нач
    если r пусто вернуть
    print_tree(r.right, уровень + 1)
    вывести отступы * уровень + r.data
    print_tree(r.left, уровень + 1)
кон

// Основная программа
нач
    установить локаль
    запросить ввод чисел для построения дерева
    разбить ввод на токены
    пока токен существует
        если токен "stop" или "in the end" тогда выйти
        если не число - вывести ошибку и пропустить
        иначе преобразовать токен в число D
        root := CreateTree(root, D)
    завершить цикл
    вывести построенное дерево

    запросить ввод чисел для подсчёта вхождений
    пока не получен стоп-команда
        ввод строки
        если строка "stop" или "in the end" - выход

```

если не число - вывести сообщение и продолжить
преобразовать в число key
count := count_occurrences(root, key)
вывести результат count

кон

3 задание

алг бинарное_дерево_без_дубликатов

дано

- пустое бинарное дерево root
- последовательность чисел для вставки (без повторов)

надо

- построить дерево поиска
- исключить вставку повторяющихся значений
- вывести дерево

нач цел

root := пусто

line := строка ввода

token := указатель для разбора строки

D := число

// проверка, является ли строчка числом

функция is_number(str)

нач

если str пустое или NULL тогда вернуть 0

для каждого символа i в str

если символ не цифра и не минус в начале — вернуть 0

конец

вернуть 1

кон

// рекурсивный поиск элемента в дереве

функция search(r, key)

нач

если r пусто или r.data = key — вернуть r

если key > r.data — вернуть search(r.right, key)

иначе — вернуть search(r.left, key)

кон


```

// рекурсивное добавление элемента без повторов
функция CreateTree(r, data)
нач
    если r пусто тогда
        выделить память под узел p
        инициализировать p.data = data, p.left = NULL, p.right = NULL
        вернуть p
    иначе
        если data > r.data тогда
            r.right := CreateTree(r.right, data)
        иначе если data < r.data тогда
            r.left := CreateTree(r.left, data)
        иначе // data равно r.data — не вставлять, дубли пропускаются
            вернуть r
    кон
    вернуть r
кон

// печать дерева с отступами
процедура print_tree(r, уровень)
нач
    если r пусто вернуть
    print_tree(r.right, уровень + 1)
    вывести пробелы * уровень + r.data
    print_tree(r.left, уровень + 1)
кон

// основная программа
нач
    установить локаль
    вывести приглашение для ввода чисел
    ввести строку line
    разбить line на токены
    для каждого токена
        если токен "stop" или "in the end" — выход
        если не число — вывести ошибку и пропустить
        иначе D := преобразовать в число
        если search(root, D) ≠ пусто — вывести ошибку про дубликат
        иначе root := CreateTree(root, D)
    вывести построенное дерево
кон

```

4 задание

алг бинарное_дерево_без_дубликатов

дано

- пустое бинарное дерево root
- последовательность чисел для вставки (без повторов)

надо

- построить дерево поиска
- исключить вставку повторяющихся значений
- вывести дерево

нач цел

root := пусто

line := строка ввода

token := указатель для разбора строки

D := число

// проверка, является ли строчка числом

функция is_number(str)

нач

если str пустое или NULL тогда вернуть 0

для каждого символа i в str

если символ не цифра и не минус в начале — вернуть 0

конец

вернуть 1

кон

// рекурсивный поиск элемента в дереве

функция search(r, key)

нач

если r пусто или r.data = key — вернуть r

если key > r.data — вернуть search(r.right, key)

иначе — вернуть search(r.left, key)

кон

// рекурсивное добавление элемента без повторов

функция CreateTree(r, data)

нач

если r пусто тогда

выделить память под узел p

инициализировать p.data = data, p.left = NULL, p.right = NULL

```

    вернуть r
  иначе
    если data > r.data тогда
      r.right := CreateTree(r.right, data)
    иначе если data < r.data тогда
      r.left := CreateTree(r.left, data)
    иначе // data равно r.data — не вставлять, дубли пропускаются
      вернуть r
  кон
  вернуть r
кон

// печать дерева с отступами
процедура print_tree(r, уровень)
нач
  если r пусто вернуть
  print_tree(r.right, уровень + 1)
  вывести пробелы * уровень + r.data
  print_tree(r.left, уровень + 1)
кон

// основная программа
нач
  установить локаль
  вывести приглашение для ввода чисел
  ввести строку line
  разбить line на токены
  для каждого токена
    если токен "stop" или "in the end" — выход
    если не число — вывести ошибку и пропустить
    иначе D := преобразовать в число
    если search(root, D) ≠ пусто — вывести ошибку про дубликат
    иначе root := CreateTree(root, D)
  вывести построенное дерево
кон

```

Листинг

1-ое задание

```
#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
#include <string.h>
#include <ctype.h>

struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

struct Node* root = NULL;

int is_number(const char* str) {
    if (str == NULL || *str == '\\0') return 0;
    for (int i = 0; str[i] != '\\0'; i++) {
        if (!isdigit(str[i]) && !(i == 0 && str[i] == '-')) {
            return 0;
        }
    }
    return 1;
}

struct Node* CreateTree(struct Node* r, int data) {
    if (r == NULL) {
        r = (struct Node*)malloc(sizeof(struct Node));
        if (r == NULL) {
            printf("Ошибка выделения памяти");
        }
    }
}
```

```

        exit(0);
    }
    r->data = data;
    r->left = NULL;
    r->right = NULL;
    return r;
}
if (data > r->data)
    r->right = CreateTree(r->right, data);
else
    r->left = CreateTree(r->left, data);
return r;
}

struct Node* search(struct Node* r, int key) {
    if (r == NULL || r->data == key)
        return r;
    if (key > r->data)
        return search(r->right, key);
    else
        return search(r->left, key);
}

void print_tree(struct Node* r, int l) {
    if (r == NULL) return;
    print_tree(r->right, l + 1);
    for (int i = 0; i < l; i++) printf(" ");
    printf("%d\n", r->data);
    print_tree(r->left, l + 1);
}

int main() {

```

```

setlocale(LC_ALL, "");
char line[1024];
char* token;
int D;

printf("Введите числа для построения дерева через
пробел.\n");

printf("Enter для завершения ввода.\n\n");

if (fgets(line, sizeof(line), stdin)) {
    token = strtok(line, " \t\n");
    while (token != NULL) {
        if (strcmp(token, "stop") == 0 || strcmp(token, "in
the end") == 0)
            break;
        if (!is_number(token)) {
            printf("Ошибка: '%s' не число, пропускаем.\n",
token);
        }
        else {
            D = atoi(token);
            root = CreateTree(root, D);
        }
        token = strtok(NULL, " \t\n");
    }
}

printf("Построенное дерево:\n");
print_tree(root, 0);
printf("\n");

printf("Для поиска введите значения по одному (in the end
для завершения):\n");

```

```

while (1) {
    printf("Введите значение для поиска: ");
    if (!fgets(line, sizeof(line), stdin)) break;
    line[strcspn(line, "\r\n")] = 0;

    if (strcmp(line, "stop") == 0 || strcmp(line, "in the
end") == 0)
        break;

    if (!is_number(line)) {
        printf("its starts with\n");
        continue;
    }
    D = atoi(line);
    struct Node* res = search(root, D);
    if (res != NULL)
        printf("Элемент %d найден в дереве.\n", D);
    else
        printf("Элемент %d не найден в дереве.\n", D);
}

return 0;
}

```

2-ое задание

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
#include <string.h>
#include <ctype.h>

struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

struct Node* root = NULL;

int is_number(const char* str) {
    if (str == NULL || *str == '\\0') return 0;
    for (int i = 0; str[i] != '\\0'; i++) {
        if (!isdigit(str[i]) && !(i == 0 && str[i] == '-')) {
            return 0;
        }
    }
    return 1;
}

struct Node* CreateTree(struct Node* r, int data) {
    if (r == NULL) {
        r = (struct Node*)malloc(sizeof(struct Node));
        if (r == NULL) {
            printf("Ошибка выделения памяти");
            exit(0);
        }
    }
}
```



```

        }

        r->data = data;
        r->left = NULL;
        r->right = NULL;
        return r;
    }

    if (data > r->data)
        r->right = CreateTree(r->right, data);
    else
        r->left = CreateTree(r->left, data);
    return r;
}

int count_occurrences(struct Node* r, int key) {
    if (r == NULL) return 0;
    int count = (r->data == key) ? 1 : 0;
    return count + count_occurrences(r->left, key) +
        count_occurrences(r->right, key);
}

void print_tree(struct Node* r, int l) {
    if (r == NULL) return;
    print_tree(r->right, l + 1);
    for (int i = 0; i < l; i++) printf(" ");
    printf("%d\n", r->data);
    print_tree(r->left, l + 1);
}

int main() {
    setlocale(LC_ALL, "");
    char line[1024];
    char* token;

```

```

int D;

printf("Введите числа для построения дерева через
пробел.\n");

printf("Введите 'stop' или 'in the end' для завершения
ввода.\n\n");

if (fgets(line, sizeof(line), stdin)) {
    token = strtok(line, " \t\n");
    while (token != NULL) {
        if (strcmp(token, "stop") == 0 || strcmp(token, "in
the end") == 0)
            break;
        if (!is_number(token)) {
            printf("Ошибка: '%s' не число, пропускаем.\n",
token);
        }
        else {
            D = atoi(token);
            root = CreateTree(root, D);
        }
        token = strtok(NULL, " \t\n");
    }
}

printf("Построенное дерево:\n");
print_tree(root, 0);
printf("\n");

printf("Введите значения для подсчёта вхождений по одному in
the end для завершения:\n");

while (1) {
    printf("Введите число: ");

```

```

        if (!fgets(line, sizeof(line), stdin)) break;
        line[strcspn(line, "\r\n")] = 0;

        if (strcmp(line, "stop") == 0 || strcmp(line, "in the
end") == 0)
            break;

        if (!is_number(line)) {
            printf("in the end.\n");
            continue;
        }
        int key = atoi(line);
        int count = count_occurrences(root, key);
        printf("Элемент %d встречается в дереве %d раз(a).\n",
key, count);
    }

    return 0;
}

```

3-ие задание

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
#include <string.h>
#include <ctype.h>

struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

struct Node* root = NULL;

int is_number(const char* str) {
    if (str == NULL || *str == '\\0') return 0;
    for (int i = 0; str[i] != '\\0'; i++) {
        if (!isdigit(str[i]) && !(i == 0 && str[i] == '-')) {
            return 0;
        }
    }
    return 1;
}

struct Node* search(struct Node* r, int key) {
    if (r == NULL || r->data == key)
        return r;
    if (key > r->data)
        return search(r->right, key);
    else
```

```

        return search(r->left, key);
    }

struct Node* CreateTree(struct Node* r, int data) {
    if (r == NULL) {
        r = (struct Node*)malloc(sizeof(struct Node));
        if (r == NULL) {
            printf("Ошибка выделения памяти");
            exit(0);
        }
        r->data = data;
        r->left = NULL;
        r->right = NULL;
        return r;
    }
    if (data > r->data)
        r->right = CreateTree(r->right, data);
    else if (data < r->data)
        r->left = CreateTree(r->left, data);
    return r;
}

void print_tree(struct Node* r, int l) {
    if (r == NULL) return;
    print_tree(r->right, l + 1);
    for (int i = 0; i < l; i++) printf(" ");
    printf("%d\n", r->data);
    print_tree(r->left, l + 1);
}

int main() {
    setlocale(LC_ALL, "");

```

```

char line[1024];

char* token;

int D;


printf("Введите числа для построения дерева без дубликатов
через пробел.\n");

printf("enter для продолжения\n\n");


if (fgets(line, sizeof(line), stdin)) {
    token = strtok(line, " \t\n");
    while (token != NULL) {
        if (strcmp(token, "stop") == 0 || strcmp(token, "in
the end") == 0)
            break;
        if (!is_number(token)) {
            printf("Ошибка: '%s' не число, пропускаем.\n",
token);
        }
        else {
            D = atoi(token);
            if (search(root, D) != NULL) {
                printf("Ошибка: число %d уже существует в
дереве. Пропускаем.\n", D);
            }
            else {
                root = CreateTree(root, D);
            }
        }
        token = strtok(NULL, " \t\n");
    }
}


printf("Построенное дерево:\n");

```

```
    print_tree(root, 0);

    return 0;
}
```

4-ое задание

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>
#include <string.h>
#include <ctype.h>
#include <math.h>

struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

struct Node* root = NULL;

int is_number(const char* str) {
    if (str == NULL || *str == '\\0') return 0;
    for (int i = 0; str[i] != '\\0'; i++) {
        if (!isdigit(str[i]) && !(i == 0 && str[i] == '-')) {
            return 0;
        }
    }
    return 1;
}

struct Node* CreateTree(struct Node* r, int data) {
    if (r == NULL) {
        r = (struct Node*)malloc(sizeof(struct Node));
        if (r == NULL) {
            printf("Ошибка выделения памяти");
        }
    }
}
```



```

        exit(0);
    }
    r->data = data;
    r->left = NULL;
    r->right = NULL;
    return r;
}
if (data > r->data)
    r->right = CreateTree(r->right, data);
else
    r->left = CreateTree(r->left, data);
return r;
}

```

```

struct Node* search(struct Node* r, int key) {
    if (r == NULL || r->data == key)
        return r;
    if (key > r->data)
        return search(r->right, key);
    else
        return search(r->left, key);
}

```

```

void print_tree(struct Node* r, int l) {
    if (r == NULL) return;
    print_tree(r->right, l + 1);
    for (int i = 0; i < l; i++) printf(" ");
    printf("%d\n", r->data);
    print_tree(r->left, l + 1);
}

```

// Функция вычисления высоты дерева

```

int height(struct Node* node) {
    if (node == NULL) return 0;
    int left_h = height(node->left);
    int right_h = height(node->right);
    return (left_h > right_h ? left_h : right_h) + 1;
}

// Функция подсчёта количества узлов
int count_nodes(struct Node* node) {
    if (node == NULL) return 0;
    return 1 + count_nodes(node->left) + count_nodes(node->right);
}

int main() {
    setlocale(LC_ALL, "");
    char line[1024];
    char* token;
    int D;

    printf("Введите числа для построения дерева через пробел.\n");

    printf("Введите 'stop' или 'in the end' для завершения ввода.\n\n");

    if (fgets(line, sizeof(line), stdin)) {
        token = strtok(line, " \t\n");
        while (token != NULL) {
            if (strcmp(token, "stop") == 0 || strcmp(token, "in the end") == 0)
                break;
            if (!is_number(token)) {
                printf("Ошибка: '%s' не число, пропускаем.\n",
token);
            }
        }
    }
}

```

```

        }
        else {
            D = atoi(token);
            root = CreateTree(root, D);
        }
        token = strtok(NULL, " \t\n");
    }
}

printf("Построенное дерево:\n");
print_tree(root, 0);
printf("\n");

// Оценка сложности поиска
int h = height(root);
int n = count_nodes(root);
double log_n = n > 0 ? log2(n) : 0;

printf("Количество узлов: %d\n", n);
printf("Высота дерева: %d\n", h);
printf("Логарифм по основанию 2 от количества узлов:
%.2f\n", log_n);

if (h <= log_n * 1.5)
    printf("Дерево приблизительно сбалансировано, время
поиска ~ O(log n)\n");
else
    printf("Дерево несбалансировано, время поиска может быть
ближе к O(n)\n");

printf("\nДля поиска введите значения по одному (stop или in
the end для завершения):\n");

```

```

while (1) {
    printf("Введите значение для поиска: ");
    if (!fgets(line, sizeof(line), stdin)) break;
    line[strcspn(line, "\r\n")] = 0;

    if (strcmp(line, "stop") == 0 || strcmp(line, "in the
end") == 0)
        break;

    if (!is_number(line)) {
        printf("Ошибка: введите число или 'stop'/'in the
end'.\n");
        continue;
    }
    D = atoi(line);
    struct Node* res = search(root, D);
    if (res != NULL)
        printf("Элемент %d найден в дереве.\n", D);
    else
        printf("Элемент %d не найден в дереве.\n", D);
}

return 0;
}

```

Результат работы программы

1 задание

```
Консоль отладки Microsoft Visual Studio
Введите числа для построения дерева через пробел.
Enter для завершения ввода.

1 2 3 4 5 6
Построенное дерево:
      6
     5
    4
   3
  2
 1

Для поиска введите значения по одному (in the end для завершения):
Введите значение для поиска: 2
its starts with
Введите значение для поиска: 3
Элемент 3 найден в дереве.
Введите значение для поиска: 2
Элемент 2 найден в дереве.
Введите значение для поиска: 9
its starts with
Введите значение для поиска: 9
Элемент 9 не найден в дереве.
Введите значение для поиска: in the end

C:\Users\amaka\source\repos\ConsoleApplication1\x64\Debug\ConsoleApplication1.exe (процесс 14580) завершил работу с кодом 0 (0x0).
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" ->"Параметры" ->"Отладка" -> "Автоматически закрыть консоль при остановке отладки".
```

2 задание

```
Консоль отладки Microsoft Visual Studio
Введите числа для построения дерева через пробел.
Введите 'stop' или 'in the end' для завершения ввода.

1 22 33 40 133
Построенное дерево:
      133
     40
    33
   22
  1

Введите значения для подсчёта вхождений по одному in the end для завершения:
Введите число: 22
Элемент 22 встречается в дереве 1 раз(a).
Введите число: 33
Элемент 33 встречается в дереве 1 раз(a).
Введите число: 3
Элемент 3 встречается в дереве 0 раз(a).
Введите число: in the edn
in the end.
Введите число: in the end

C:\Users\amaka\source\repos\ConsoleApplication1\x64\Debug\ConsoleApplication1.exe (процесс 22620) завершил работу с кодом 0 (0x0).
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" ->"Параметры" ->"Отладка" -> "Автоматически закрыть консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно:
```

3 задание

```
Консоль отладки Microsoft Visual Studio
Введите числа для построения дерева без дубликатов через пробел.
enter для продолжения
1 2 2 4 56 659
Ошибка: число 2 уже существует в дереве. Пропускаем.
Построенное дерево:
      659
     /
    56
   /
  4
 /
2
/
1

C:\Users\amaka\source\repos\ConsoleApplication1\x64\Debug\ConsoleApplication1.exe (процесс 16044) завершил работу с кодом 0 (0x0).
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" ->"Параметры" ->"Отладка" -> "Автоматически закрыть консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно: _
```

4 задание

```
C:\Users\amaka\source\repos\ConsoleApplication1\x64\Debug\ConsoleApplication1.exe
Введите числа для построения дерева через пробел.
Введите 'stop' или 'in the end' для завершения ввода.
1 2 344 543 1212
Построенное дерево:
      1212
     /
    543
   /
  344
 /
2
/
1

Количество узлов: 5
Высота дерева: 5
Логарифм по основанию 2 от количества узлов: 2,32
Дерево несбалансировано, время поиска может быть ближе к O(n)

Для поиска введите значения по одному (stop или in the end для завершения):
Введите значение для поиска: 344
Элемент 344 найден в дереве.
Введите значение для поиска: 3
Элемент 3 не найден в дереве.
Введите значение для поиска: _
```

Вывод

Оба варианта реализации бинарного дерева поиска демонстрируют классические алгоритмы работы с BST. Первая версия показывает базовый принцип работы с простой вставкой, но допускает дубликаты, что не соответствует строгому определению BST. Вторая версия исправляет этот недостаток за счет двойного обхода при вставке, обеспечивая уникальность элементов. Если дерево сбалансировано, то асимптотическая сложность основных операций (вставки, удаления и поиска) составляет $O(\log_2 n)$, однако в случае несбалансированной структуры она возрастает до $O(n)$.